

Outfit Planner — Sequential Task List

Cross-referenced against monolithic-architecture-plan.md and full project scan. = Done | = Partially done / has bugs | = Not started

Phase 1: Backend Foundation

1.1 Domain Layer

- ☒ Create **User** entity inheriting from IdentityUser with style profile & preferences
- ☒ Create **ClothingItem** entity with tags and metadata
- ☒ Create **Outfit** entity with items and relationships
- ☒ Create **ValidationPoll** entity for social features
- ☒ Create **WearEvent** entity for tracking
- ☒ Define all enums: **ClothingType**, **OccasionType**, **Season**, **StylePreference**, **PrivacyLevel**, **FabricType**, **PollStatus**, **OutfitStatus**, **ItemRole**
- ☒ Create value objects: **Money**, **Color**, **Temperature**, **ValueObject** base
- ☒ Define repository interfaces (15 interfaces in **Contracts/Persistence**)

1.2 Persistence / Infrastructure Layer

- ☒ Configure SQL Server **AppDbContext** with EF Core
- ☒ Create entity configurations with Fluent API (8 config files)
- ☒ Implement generic repository pattern (**GenericRepository.cs**)
- ☒ Implement specific repositories (15 repositories)
- ☒ Configure database migrations
- ☒ Implement JWT authentication service
- ☒ Implement **UnitOfWork** pattern
- ☐ **Task 1:** Implement **ExceptionMiddleware** (currently empty stub)
- ☐ **Task 2:** Implement **RequestLoggingMiddleware** (currently empty stub)
- ☐ **Task 3:** Implement image storage service (**ImageProcessingService**)
- ☐ **Task 4:** Implement email service (**EmailService**)

1.3 Application Layer — CQRS (This is the biggest gap)

- ☒ Set up MediatR for CQRS (**DependencyInjection.cs**)
- ☒ Configure AutoMapper profiles (partially — needs more maps)
- ☒ Create DTOs: **ClothingItemDto**, **ClothingTagDto**, **ClothingItemListDto**

Bug Fixes (Do These First):

- ☐ **Task 5:** Fix **GetClothingItemRequestHandler** — add **async** keyword to **Handle** method

- ☐ **Task 6:** Fix duplicate class name — rename class in `GetClothingItemListRequest.cs` from `GetClothingItemRequest` to `GetClothingItemListRequest`
- ☐ **Task 7:** Fix `GetClothingItemListRequest` return type — should return `List<ClothingItemDto>` not `ClothingItemDto`
- ☐ **Task 8:** Add missing `using` statements in handler and request files
- ☐ **Task 9:** Fix `WardrobeController.GetById` — references non-existent `GetClothingItemByIdRequest`, should be `GetClothingItemRequest`

Wardrobe Queries:

- ☐ **Task 10:** Fix `GetClothingItemRequestHandler` (get single item by ID — exists but buggy)
- ☐ **Task 11:** Create `GetClothingItemListRequestHandler` (get all items for a user)
- ☐ **Task 12:** Create `GetClothingItemsByCategoryRequest` + Handler

Wardrobe Commands:

- ☐ **Task 13:** Create `CreateClothingItemCommand` + `CreateClothingItemCommandHandler`
- ☐ **Task 14:** Create `UpdateClothingItemCommand` + `UpdateClothingItemCommandHandler`
- ☐ **Task 15:** Create `DeleteClothingItemCommand` + `DeleteClothingItemCommandHandler` (soft delete)
- ☐ **Task 16:** Create `RecordWearCommand` + `RecordWearCommandHandler`

Wardrobe DTOs & Validators:

- ☐ **Task 17:** Create `CreateClothingItemDto` (exists in DTOs folder but mapping is commented out)
- ☐ **Task 18:** Create `UpdateClothingItemDto` (exists but mapping is commented out)
- ☐ **Task 19:** Create FluentValidation validators for Create/Update commands
- ☐ **Task 20:** Update `MappingProfile.cs` — uncomment and complete all mappings

Outfit Queries & Commands:

- ☐ **Task 21:** Create `OutfitDto`, `OutfitListDto`, `OutfitItemDto`
- ☐ **Task 22:** Create `GetOutfitsRequest` + Handler (get all outfits for user)
- ☐ **Task 23:** Create `GetOutfitByIdRequest` + Handler
- ☐ **Task 24:** Create `CreateOutfitCommand` + Handler
- ☐ **Task 25:** Create `UpdateOutfitCommand` + Handler
- ☐ **Task 26:** Create `DeleteOutfitCommand` + Handler
- ☐ **Task 27:** Create `RecordOutfitWearCommand` + Handler
- ☐ **Task 28:** Create `GenerateOutfitSuggestionsQuery` + Handler (outfit algorithm placeholder)

Social Queries & Commands:

- ☐ **Task 29:** Create `ValidationPollDto`, `PollOptionDto`, `VoteDto`
- ☐ **Task 30:** Create `GetPollsRequest` + Handler
- ☐ **Task 31:** Create `GetPollByIdRequest` + Handler
- ☐ **Task 32:** Create `CreatePollCommand` + Handler
- ☐ **Task 33:** Create `VoteOnPollCommand` + Handler

Weather Queries:

- ☐ **Task 34:** Create `GetCurrentWeatherQuery` + Handler (mock/placeholder service)
- ☐ **Task 35:** Create `GetWeatherForecastQuery` + Handler

1.4 API Layer — Controllers

- ☒ Configure JWT Bearer authentication in `Program.cs`
- ☒ Create `AuthController` (register, login, refresh) — working

WardrobeController (currently almost entirely commented out):

- ☐ **Task 36:** Uncomment & wire `GET /api/wardrobe` — `GetAll` endpoint using `MediatR`
- ☐ **Task 37:** Fix & wire `GET /api/wardrobe/{id}` — `GetById` endpoint
- ☐ **Task 38:** Uncomment & wire `GET /api/wardrobe/category/{category}` — `GetByCategory`
- ☐ **Task 39:** Uncomment & wire `POST /api/wardrobe` — `Create` endpoint
- ☐ **Task 40:** Uncomment & wire `PUT /api/wardrobe/{id}` — `Update` endpoint
- ☐ **Task 41:** Uncomment & wire `DELETE /api/wardrobe/{id}` — `Delete` endpoint
- ☐ **Task 42:** Uncomment & wire `POST /api/wardrobe/{id}/wear` — `RecordWear` endpoint
- ☐ **Task 43:** Remove the old `MapToDto` private method (replaced by `AutoMapper`)

OutfitsController (currently empty):

- ☐ **Task 44:** Implement `GET /api/outfits` — Get all user outfits
- ☐ **Task 45:** Implement `GET /api/outfits/{id}` — Get outfit by ID
- ☐ **Task 46:** Implement `POST /api/outfits/generate` — Generate suggestions
- ☐ **Task 47:** Implement `POST /api/outfits` — Save new outfit
- ☐ **Task 48:** Implement `PUT /api/outfits/{id}` — Update outfit
- ☐ **Task 49:** Implement `DELETE /api/outfits/{id}` — Delete outfit
- ☐ **Task 50:** Implement `GET /api/outfits/today` — Today's suggestion
- ☐ **Task 51:** Implement `POST /api/outfits/{id}/wear` — Record outfit wear

SocialController (currently empty):

- ☐ **Task 52:** Implement GET /api/social/polls — Get user polls
- ☐ **Task 53:** Implement GET /api/social/polls/{id} — Get poll by ID
- ☐ **Task 54:** Implement POST /api/social/polls — Create poll
- ☐ **Task 55:** Implement POST /api/social/polls/{id}/vote — Vote on poll
- ☐ **Task 56:** Implement GET /api/social/trends/local — Local trends (placeholder)

WeatherController (currently empty):

- ☐ **Task 57:** Implement GET /api/weather/current — Current weather
- ☐ **Task 58:** Implement GET /api/weather/forecast — Weather forecast

Middleware & Cross-Cutting:

- ☐ **Task 59:** Implement `ExceptionHandler` with global error handling
 - ☐ **Task 60:** Implement `RequestLoggingMiddleware` with Serilog
 - ☐ **Task 61:** Configure Swagger/OpenAPI documentation
 - ☐ **Task 62:** Add `FluentValidation` pipeline behavior in `MediatR`
-

Phase 2: Frontend Foundation

2.1 Core Module

- ☒ Angular project scaffolded with Clean Architecture structure
- ☒ Guards folder created (auth guard exists)
- ☒ Interceptors folder created (auth interceptor exists)
- ☒ Auth model defined
- ☐ **Task 63:** Review & complete `AuthGuard` implementation
- ☐ **Task 64:** Review & complete `AuthInterceptor` (JWT token injection, refresh logic)
- ☐ **Task 65:** Create `ErrorInterceptor` for global HTTP error handling
- ☐ **Task 66:** Review & complete `AuthService` with token storage and refresh

2.2 Domain Layer

- ☒ `ClothingItem` entity defined
- ☒ `Outfit` entity defined
- ☒ `User` entity defined
- ☒ `ClothingItemRepository` interface defined
- ☒ `OutfitRepository` interface defined
- ☒ `ClothingItemUseCases` defined
- ☐ **Task 67:** Create `ValidationPoll` entity interface
- ☐ **Task 68:** Create `WearEvent` entity interface

- ☐ **Task 69:** Create `SocialRepository` interface
- ☐ **Task 70:** Create `WeatherRepository` interface
- ☐ **Task 71:** Create `OutfitUseCases` class
- ☐ **Task 72:** Create `SocialUseCases` class

2.3 Data Layer

- ☒ `ClothingItemDataSource` created
- ☒ `ClothingItemRepositoryImpl` created
- ☐ **Task 73:** Create `OutfitDataSource` + `OutfitRepositoryImpl`
- ☐ **Task 74:** Create `SocialDataSource` + `SocialRepositoryImpl`
- ☐ **Task 75:** Create `WeatherDataSource` + `WeatherRepositoryImpl`
- ☐ **Task 76:** Create `AuthDataSource` for login/register/refresh API calls

2.4 State Management (NgRx)

- ☐ **Task 77:** Install and configure NgRx Store
- ☐ **Task 78:** Create `auth` state slice (actions, reducer, effects, selectors)
- ☐ **Task 79:** Create `wardrobe` state slice
- ☐ **Task 80:** Create `outfits` state slice
- ☐ **Task 81:** Create `social` state slice
- ☐ **Task 82:** Create `weather` state slice

Phase 3: Feature Implementation (UI Pages)

3.1 Authentication Feature

- ☒ Login page component created
- ☒ Register page component created
- ☐ **Task 83:** Wire login page to `AuthService` → backend API
- ☐ **Task 84:** Wire register page to `AuthService` → backend API
- ☐ **Task 85:** Implement token refresh mechanism
- ☐ **Task 86:** Add route guards to protected pages

3.2 Wardrobe Management Feature (UI)

- ☐ **Task 87:** Create **Wardrobe Dashboard** page (grid of clothing items with filtering)
- ☐ **Task 88:** Create **Add Clothing Item** page (form with image upload)
- ☐ **Task 89:** Create **Edit Clothing Item** page
- ☐ **Task 90:** Create **Clothing Item Detail** page
- ☐ **Task 91:** Create **Wardrobe Analytics** component (stats, most worn, least worn, etc.)
- ☐ **Task 92:** Add wardrobe routes to `app.routes.ts`

3.3 Outfit Management Feature (UI)

- ☐ **Task 93:** Create **Outfits Dashboard** page (saved outfits grid)
- ☐ **Task 94:** Create **Outfit Builder** page (drag & drop clothing items)
- ☐ **Task 95:** Create **Daily Suggestion** page (today's outfit based on weather)
- ☐ **Task 96:** Create **Weather Display** component
- ☐ **Task 97:** Add outfit routes to `app.routes.ts`

3.4 Social Validation Feature (UI)

- ☐ **Task 98:** Create **Community Feed** page (list of polls)
- ☐ **Task 99:** Create **Create Poll** page
- ☐ **Task 100:** Create **Poll Detail / Vote** page
- ☐ **Task 101:** Add social routes to `app.routes.ts`

3.5 Layout & Navigation

- ☐ **Task 102:** Create **Main Layout** component (sidebar/nav + content area)
 - ☐ **Task 103:** Create **Bottom Navigation** or **Sidebar** component
 - ☐ **Task 104:** Create **Home/Dashboard** page (overview of all features)
 - ☐ **Task 105:** Create **User Profile / Settings** page
-

Phase 4: Integration & Polish

4.1 API Integration

- ☐ **Task 106:** Connect all frontend services to backend APIs
- ☐ **Task 107:** Implement proper error handling (toasts, error pages)
- ☐ **Task 108:** Add loading states and skeleton screens
- ☐ **Task 109:** Implement optimistic updates where appropriate

4.2 UI/UX Polish

- ☐ **Task 110:** Implement responsive design (mobile-first)
- ☐ **Task 111:** Add animations and transitions
- ☐ **Task 112:** Implement accessibility features (ARIA, keyboard nav)
- ☐ **Task 113:** Create a design system (colors, typography, spacing tokens)
- ☐ **Task 114:** Dark mode support

4.3 Testing

- ☐ **Task 115:** Write unit tests for domain entities
- ☐ **Task 116:** Write unit tests for CQRS handlers
- ☐ **Task 117:** Write integration tests for API endpoints

- ☐ **Task 118:** Write Angular component tests
- ☐ **Task 119:** Write E2E tests for critical flows (login → add item → create outfit)

4.4 Documentation & Deployment

- ☐ **Task 120:** Document all API endpoints with Swagger
- ☐ **Task 121:** Create deployment guide
- ☐ **Task 122:** Create README with setup instructions
- ☐ **Task 123:** CI/CD pipeline configuration

Recommended Execution Order

[!IMPORTANT] **Work on tasks sequentially in this order.**
Each group builds on the previous one.

Order	Tasks	What You're Building
1st	Tasks 5–9	Fix all existing bugs
	Tasks 10–20	Complete Wardrobe CQRS (queries, commands, DTOs, validators)
2nd		
3rd	Tasks 36–43	Wire WardrobeController (uncomment and connect to MediatR)
4th	Tasks 59–62	Middleware & cross-cutting (error handling, logging, Swagger)
5th	Tasks 63–66	Angular core (auth guard, interceptors, auth service)
6th	Tasks 83–86	Wire auth UI to backend
7th	Tasks 77–82	NgRx state management setup
8th	Tasks 87–92	Wardrobe UI pages (the first full vertical slice!)
9th	Tasks 21–28, 44–51	Outfits backend (CQRS + controller)
10th	Tasks 93–97	Outfits UI pages
11th	Tasks 29–35, 52–58	Social + Weather backend
12th	Tasks 98–105	Social UI + layout + navigation
13th	Tasks 106–114	Integration & polish
14th	Tasks 115–123	Testing & deployment